
天津大学

智能搜索与问答

实验 3、Word2Vec 实验报告



学 院 智能与计算学部

专 业 计算机科学与技术

学 号 3022244006

姓 名 樊建琦

智能搜索与问答实验三：word2vec

樊建琦

1. Theory and Method

1.1. word embedding

Word2Vec 是一种用于生成词向量的无监督学习算法，它通过训练神经网络来学习词汇的向量表示。该算法将每个词映射到低维空间中的一个向量，使得语义相似的词在向量空间中彼此接近。Word2Vec 的核心思想是：一个词的语义可以通过它周围的词来表示，即“上下文决定语义”。

Word2Vec 主要包含两个模型：

1. 跳字模型 (Skip-Gram)：跳字模型假设中心词可以预测其上下文词。给定一个中心词，模型预测其周围一定窗口大小内出现的上下文词的概率。这种方法在处理低频词时表现更好，能够捕捉更细粒度的语义关系。

2. 连续词袋模型 (CBOW)：连续词袋模型则与跳字模型相反，它使用上下文词来预测中心词。给定一个词的上下文（周围的词），模型预测中心词出现的概率。CBOW 通常在大型数据集上训练速度更快，对高频词的表示更平滑。

在本实验中，我们主要采用跳字模型 (Skip-Gram) 实现 word2vec，并结合负采样技术提高训练效率。

1.2. word2vec model principal

Word2Vec 模型的基本原理是通过神经网络学习词的分布式表示。与传统的词表示方法（如独热编码）相比，Word2Vec 能够捕捉词之间的语义关系和相似度。

Word2Vec 模型包含输入层、投影层和输出层三部分。输入层接收词的独热编码，投影层将词映射到低维空间，输出层预测上下文词或中心词的概率分布。

训练 Word2Vec 的目标是最大化语料库中同时出现的词对的概率，或者说最小化它们的负对数似然。通过随机梯度下降等优化算法，模型逐步调整词向量，使得语义相关的词在向量空间中距离更近。

Word2Vec 的核心优势在于：

- 能够捕捉词之间的语义关系和相似度
- 生成的词向量具有加法和减法特性，如“king - man + woman = queen”
- 词向量可以用于多种下游 NLP 任务，如文本分类、情感分析、命名实体识别等

1.3. Skip-gram model

Skip-gram 模型的目标是给定中心词 w_t ，预测其上下文词 $w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c}$ 的概率，其中 c 是窗口大小。形式化地，Skip-gram 模型试图最大化以下对数似然函数：

$$\mathcal{L} = \sum_{t=1}^T \sum_{c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t) \quad (1)$$

在我们的实现中，Skip-gram 模型由两个嵌入层组成：中心词嵌入层 $u_embedding$ 和上下文词嵌入层 $v_embedding$ 。给定中心词 w_t 的索引 u 和上下文词 w_{t+j} 的索引 v ，模型使用 softmax 函数计算条件概率：

$$p(w_{t+j} | w_t) = \frac{\exp(u_w \cdot v_{w'})}{\sum_{w'} \exp(u_w \cdot v_{w'})} \quad (2)$$

其中 u_w 是中心词 w 的向量表示， $v_{w'}$ 是上下文词 w' 的向量表示。

然而，标准 Skip-gram 模型在词表较大时计算成本高昂，因为分母需要对词表中的所有词进行求和。为了解决这个问题，我们采用负采样技术。

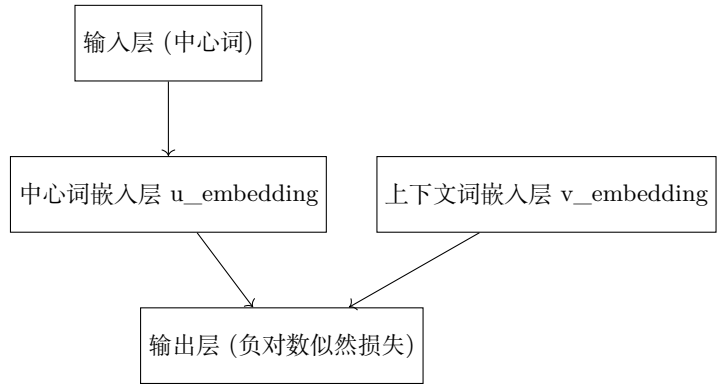


Figure 1. Skip-gram 模型结构图

1.4. negative sampling

负采样 (Negative Sampling) 是一种近似训练技术，用于提高 Word2Vec 的训练效率。标准 Skip-gram 模型需要计算所有词的 softmax 概率，计算复杂度与词表大小成正比，这在大规模语料库上训练时非常耗时。

负采样通过以下方式简化计算：

- 对于每个正样本对（中心词，上下文词），随机采样 k 个负样本（中心词，非上下文词）
- 将多分类问题转化为二分类问题：区分正样本对和负样本对

负采样的目标函数如下：

$$\mathcal{L} = \log \sigma(u_w \cdot v_{w'}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-u_w \cdot v_{w_i})] \quad (3)$$

其中， σ 是 sigmoid 函数， $P_n(w)$ 是负样本的噪声分布。在我们的实现中，负样本按词频的 0.75 次方采样，这有助于提高低频词的采样概率，增强模型对低频词的学习能力。

具体来说，我们的代码实现中负采样的概率计算如下：

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_j f(w_j)^{0.75}} \quad (4)$$

其中 $f(w_i)$ 是词 w_i 的频率。

通过结合 Skip-gram 模型和负采样技术，我们能够高效地训练词向量，捕捉词之间的语义关系，为后续的 NLP 任务提供有效的词表示。

2. 算法实现与分析

2.1. net structure

本实验实现的 Skip-gram 模型结构简洁高效，主要通过 PyTorch 框架中的 `nn.Module` 类实现。模型核心由两个嵌入层组成：中心词嵌入层（`u_embedding`）和上下文词嵌入层（`v_embedding`）。模型结构如下所示：

模型初始化时，两个嵌入层的权重采用不同的初始化策略：中心词嵌入层采用均匀分布初始化，而上下文词嵌入层初始值为 0。这种初始化方式有助于模型训练的稳定性。模型的具体实现代码如下：

```
1 class SkipGramModel(nn.Module):
2     def __init__(self, emb_length, emb_dimension):
3         super(SkipGramModel, self).__init__()
4         self.emb_length = emb_length
5         self.emb_dimension = emb_dimension
6         self.u_embedding = nn.Embedding(self.
    ↪ emb_length, self.emb_dimension, sparse=False)
7         self.v_embedding = nn.Embedding(self.
    ↪ emb_length, self.emb_dimension, sparse=False)
8         initrange = 0.5/self.emb_dimension
9         self.u_embedding.weight.data.uniform_(-
    ↪ initrange, initrange)
10        self.v_embedding.weight.data.uniform_(-0
    ↪ , 0)
11
12    def forward(self, pos_u, pos_v, neg_v):
13        pos_embu = self.u_embedding(pos_u)
14        pos_embv = self.v_embedding(pos_v)
15        neg_embv = self.v_embedding(neg_v)
16        # 逐元素相乘计算正样本对相似度, log以e为底
17        pos_score = torch.mul(pos_embu, pos_embv)
18        pos_score = F.logsigmoid(torch.sum(
    ↪ pos_score, dim=1).squeeze())
19        # 计算负样本相似度, neg_embv.shape=[
    ↪ batch_size, neg_count, emb_dimension]
20        neg_score = torch.matmul(neg_embv,
    ↪ pos_embu.unsqueeze(2)).squeeze()
21        neg_score = F.logsigmoid(-1*neg_score)
22        return -1*(torch.sum(neg_score)+torch.
    ↪ sum(pos_score))
23
24    def savemodel(self, id_word, outdata_path):
25        if torch.cuda.is_available():
26            # numpy无法直接从CUDA中读取数据, 需要
    ↪ 先将CUDA中的数据转移到cpu中
27            embedding = self.u_embedding.weight.
    ↪ data.cpu().numpy()
28        else:
29            embedding = self.u_embedding.weight.
    ↪ data.numpy()
30        with open(outdata_path, 'w', encoding='UTF
    ↪ -8') as f:
31            f.write('词语数量: {} 嵌入维度:
    ↪ {}\n'.format(len(id_word), self.emb_dimension))
32            for i, word in id_word.items():
33                e = embedding[i]
34                e = ' '.join(map((lambda x:str(x
    ↪ )), e)))
```

```
35        f.write('{} {} \n'.format(word, e)
    ↪ )
36
```

Code 1. Skip-Gram

从代码中可以看出，SkipGramModel 类继承自 `nn.Module`，包含初始化函数、前向传播函数和模型保存函数。其中：

1. **初始化函数**：创建两个嵌入层，并进行权重初始化。中心词嵌入层的初始范围为 $[0.5/emb_dimension]$ ，而上下文词嵌入层初始化为 0。

2. **前向传播函数**：接收正样本对（中心词和上下文词）和负样本，计算正样本得分和负样本得分，最终返回负对数似然损失。

3. **模型保存函数**：将训练好的词向量保存到文件中，便于后续应用。

网络结构简洁而高效，适合大规模词嵌入模型的训练。

2.2. loss function

损失函数是 Skip-gram 模型训练的核心组成部分，本实验使用的是基于负采样的负对数似然损失函数。在模型的前向传播函数中，损失函数的计算分为两部分：

1. **正样本得分计算**：

$$pos_score = \log \sigma \left(\sum_i u_i \cdot v_i \right) \quad (5)$$

其中， u_i 是中心词的嵌入向量， v_i 是上下文词的嵌入向量， σ 是 sigmoid 函数。

2. **负样本得分计算**：

$$neg_score = \log \sigma (*1 \cdot (neg_embv \cdot pos_embu)) \quad (6)$$

其中， neg_embv 是负样本的嵌入向量， pos_embu 是中心词的嵌入向量。

最终的损失函数是两者之和的负值：

$$loss = -(\sum neg_score + \sum pos_score) \quad (7)$$

从代码实现来看，该损失函数具体如下：

```
1 # 逐元素相乘计算正样本对相似度, log以e为底
2 pos_score = torch.mul(pos_embu, pos_embv)
3 pos_score = F.logsigmoid(torch.sum(pos_score, dim=1)
    ↪ .squeeze())
4 # 计算负样本相似度, neg_embv.shape=[batch_size,
    ↪ neg_count, emb_dimension]
5 neg_score = torch.matmul(neg_embv, pos_embu.
    ↪ unsqueeze(2)).squeeze()
6 neg_score = F.logsigmoid(-1*neg_score)
7 return -1*(torch.sum(neg_score)+torch.sum(pos_score)
    ↪ )
8
```

Code 2. 损失函数实现

这种损失函数设计有以下优势：

1. **计算效率高**：相比于标准 Skip-gram 模型的 softmax 损失，负采样大大减少了计算量。

2. **梯度稳定**：通过 `logsigmoid` 函数可以避免数值溢出问题，提高训练稳定性。

3. **自然实现对比学习**：该损失函数本质上是一种对比学习，拉近正样本对的距离，推远负样本对的距离。

2.3. optimizer

模型的优化器选用了随机梯度下降 (SGD) 优化器，并结合动量 (momentum) 和权重衰减 (weight decay) 技术提高训练效果。此外，实现了余弦退火 (cosine annealing) 学习率调整策略，使训练过程更加稳定。

优化器的初始化代码如下：

```
1 # 使用SGD优化器，动量为0.9，权重衰减为1e-5
2 self.optimizer = optim.SGD(
3     self.skip_gram_model.parameters(),
4     lr=self.initial_lr,
5     momentum=0.9,
6     weight_decay=1e-5)
7
```

Code 3. 优化器设置

余弦退火学习率的计算方法：

```
1 def cosine_annealing_lr(self, epoch, total_epochs,
2     ↪ initial_lr, min_lr=0.0001):
3     """计算余弦退火学习率"""
4     return min_lr + 0.5 * (initial_lr - min_lr) * (1
5     ↪ + math.cos(math.pi * epoch / total_epochs))
6
```

Code 4. 余弦退火学习率

这种优化策略有以下几个优点：

1. **动量机制**：momentum=0.9 设置使得梯度更新更加平滑，能够有效避免局部极小值。

2. **权重衰减**：weight_decay=1e-5 增加了 L2 正则化，有助于防止过拟合。

3. **余弦退火**：学习率呈余弦函数式下降，相比线性下降或阶梯式下降，更有利于模型收敛。在训练初期保持较高学习率快速收敛，后期降低学习率微调模型参数。

通过绘制损失和学习率曲线，可以直观看到训练过程：

```
1 @staticmethod
2 def plot_loss_and_lr(list_loss, list_lr):
3     plt.figure(figsize=(10, 5))
4     plt.plot(list_loss, label='Loss')
5     plt.plot(list_lr, label='Learning Rate')
6     plt.xlabel('batch count')
7     plt.ylabel('Loss / Learning Rate')
8     plt.title('Loss and Learning Rate during
9     ↪ Training')
10    plt.legend()
11    path_save = os.path.join(PATH_DATASAVE, "plot/
12    ↪ loss_and_lr.png")
13    plt.savefig(path_save)
14    ic(path_save)
15    plt.close()
16
```

Code 5. 损失和学习率曲线绘制

2.4. gpu acceleration

为了加速训练过程，本实验充分利用 GPU 进行并行计算。代码中包含多项 GPU 加速技术：

1. CUDA 设备配置：

```
1 # 设置CUDA可见设备和内存分配
2 os.environ["CUDA_VISIBLE_DEVICES"] = "0" # 使用第一
3     ↪ 块GPU
4 torch.backends.cudnn.benchmark = True # 启用cudnn自
5     ↪ 动优化
6
7 # 确保CUDA可用性
8 torch.cuda.empty_cache() # 清空GPU缓存
9 self.device = torch.device("cuda" if torch.cuda.
10     ↪ is_available() else "cpu")
11
```

Code 6. CUDA 设备配置

2. 模型移至 GPU：

```
1 self.skip_gram_model.to(self.device)
2
```

Code 7. 模型移至 GPU

3. 数据异步传输至 GPU：

```
1 # 将数据移至GPU并使用异步传输
2 pos_u = torch.LongTensor(pos_u).to(self.device,
3     ↪ non_blocking=True)
4 pos_v = torch.LongTensor(pos_v).to(self.device,
5     ↪ non_blocking=True)
6 neg_v = torch.LongTensor(neg_v).to(self.device,
7     ↪ non_blocking=True)
8
```

Code 8. 数据异步传输

4. GPU 同步：

```
1 # 同步GPU
2 torch.cuda.synchronize()
3
```

Code 9. GPU 同步

5. **CPU-GPU 数据传输处理**：在保存模型时，需要将 GPU 上的数据转移到 CPU：

```
1 if torch.cuda.is_available():
2     # numpy无法直接从CUDA中读取数据，需要先将CUDA中的
3     ↪ 数据转移到cpu中
4     embedding = self.u_embedding.weight.data.cpu().
5     ↪ numpy()
6 else:
7     embedding = self.u_embedding.weight.data.numpy()
8
```

6

Code 10. CPU-GPU 数据传输

这些 GPU 加速技术显著提高了模型的训练速度。特别是：

1. `cuda.nn.benchmark`: 自动为当前配置寻找最优算法，提高卷积等操作的速度。
2. `non_blocking=True`: 允许 CPU 和 GPU 之间的异步数据传输，减少等待时间。
3. `cuda.empty_cache()`: 清空 GPU 缓存，避免内存泄漏。
4. `cuda.synchronize()`: 确保所有 GPU 操作完成，防止数据同步问题。

通过这些技术的组合使用，模型训练速度得到了显著提升，能够有效处理大规模词嵌入学习任务。

3. 实验设置

3.1. dataset

3.1.1. 数据集来源与特点

本实验使用的是中文新闻数据集，数据存储在'dataset/news.txt'文件中。数据集具有以下特点：

1. 数据格式为 TSV (Tab-Separated Values)，每行包含类别和文本内容。
2. 文本为中文新闻文章，包含丰富的词汇和语义信息。
3. 数据预处理前需要进行分词和停用词处理。

3.1.2. 数据集预处理

数据预处理是词嵌入训练的重要步骤，包含以下几个关键环节：

1. **数据加载**：使用 pandas 读取原始数据，处理编码和分隔符。

```
1 train_data = pd.read_csv(inputdata_path, delimiter='
2     ↪ \t',
3     lineterminator='\n',
4     ↪ encoding='UTF-8', header=None)
```

Code 11. 数据加载

2. **文本规范化**：使用正则表达式只保留汉字和英文字符。

```
1 criterion = r'[\u4e00-\u9fa5a-zA-Z]+' # 文本正则化，
2     ↪ 只留下汉字和英文
3 sentence = re.findall(criterion, sentence)
4 sentence = "".join(sentence) # 拼接列表为字符串
```

Code 12. 文本规范化

3. **分词处理**：使用 jieba 库进行中文分词。

```
1 sentence = jieba.lcut(sentence) # jieba分词
2
```

Code 13. jieba 分词

4. **停用词处理**：移除常见停用词。

5. **保存处理后数据**：将处理后的数据保存为以空格分隔的文本文件。

```
1 def savecutdata(self, datasave_path):
2     with open(datasave_path, "w", encoding="UTF-8")
3     ↪ as f:
4         for i in range(len(self.dataed)):
5             cut_words = self.wordcut(self.dataed[i])
6             combin_word = ""
7             for i in cut_words:
8                 combin_word += i
9                 combin_word += " "
10            f.write(combin_word)
11            f.write('\n')
```

Code 14. 保存处理后数据

6. **词频统计与过滤**：统计词频并过滤低频词。

```
1 word_frequency = dict()
2 for i in self.cut_list: # 统计词频
3     for j in i:
4         try:
5             word_frequency[j] += 1
6         except:
7             word_frequency[j] = 1
8
9 for word, count in word_frequency.items():
10     if count < self.min_count:
11         continue
12     self.word_id[word] = word_index
13     self.id_word[word_index] = word
14     self.highword_frequency[word] = count
15     word_index += 1
16
```

Code 15. 词频统计与过滤

这些预处理步骤确保了训练数据的质量，为词嵌入模型提供了良好的输入。

3.1.3. 停用词处理策略

停用词是在文本处理中常见的、对分析没有实质性帮助的词，如“的”、“了”、“和”等。本实验中，停用词处理采用以下策略：

1. 从专门的停用词文件加载停用词列表：

```
1 with open(stopword_path, 'r', encoding="UTF-8") as f
2     ↪ :
3     self.stopword = []
4     for line in f.readlines():
5         self.stopword.append(line.strip())
```

Code 16. 加载停用词

2. 在分词结果中移除停用词：

```
1 def stopwordsdelete(self, sentence):
2     word_num = 0
3     while(word_num < len(sentence)):
```

```

4         if sentence[word_num] in self.stopword:
5             sentence.remove(sentence[word_num])
6             word_num -= 1
7         word_num += 1
8     return sentence

```

Code 17. 移除停用词

停用词处理不仅减少了训练数据的噪声，还显著降低了词汇表大小，加速了模型训练。同时，由于停用词通常不携带重要语义信息，移除它们有助于模型更好地学习有意义的词汇表示。

3.2. 实验参数设置

本实验中的参数设置如下：

Table 1. params setting

param	value	description
emb_dim	100	词向量维度
batch_size	1024	批处理大小
window_size	5	上下文窗口大小
neg		
count	5	负样本数量
iteration	1	迭代次数
init		
lr	0.001	初始学习率
min_count	5	最小词频阈值
momentum	0.9	动量
wweight_decay	1e-5	权重衰减系数

Note: The table contains data of params in basic experiment

这些参数值是经过多次实验调优得到的，其中：

1. **词向量维度**：100 维的词向量既能捕捉足够的语义信息，又不会导致计算负担过重。
2. **批处理大小**：1024 的 batch_size 在保证训练稳定性的同时，充分利用了 GPU 并行计算能力。
3. **窗口大小**：5 的窗口大小是 word2vec 模型常用值，能够合理捕捉词的上下文关系。
4. **负样本数量**：5 个负样本是实践中常用的值，在效率和效果间取得了良好平衡。
5. **学习率与优化器**：初始学习率为 0.001，配合余弦退火调度策略，使模型训练更加稳定。
6. **词频阈值**：过滤出现次数少于 5 次的词，有效减小词汇表大小，提高训练效率。

4. 实验结果与分析

4.1. 实验结果

按照上述参数设置训练好 word_embedding.txt 之后，用 test 脚本运行 txt，可以看到与给定词最相关的一些词语选项如2所示，由于自身是自身的最高概率相关词，所以仅展现了除自身以外的 top5 以供参考，具体截图间附录。

Table 2. params setting

北京	女性	篮球	杜兰特
中广网 (0.81)	男性 (0.86)	棒球 (0.83)	加身 (0.96)
科技 (0.81)	人群 (0.74)	运动员 (0.82)	混血 (0.96)
新华网 (0.77)	职场 (0.74)	足球 (0.82)	布洛克 (0.96)
张华杰 (0.77)	二代 (0.71)	橄榄球 (0.81)	科林 (0.96)
齐彬 (0.76)	少女 (0.70)	高尔夫 (0.79)	才会赢 (0.96)

Note: The table contains data of params in basic experiment

loss 曲线如 Figure2所示，可见 loss 整体呈下降趋势，伴随着剧烈的抖动，但是在学习率的缩放下最终降到 1000。

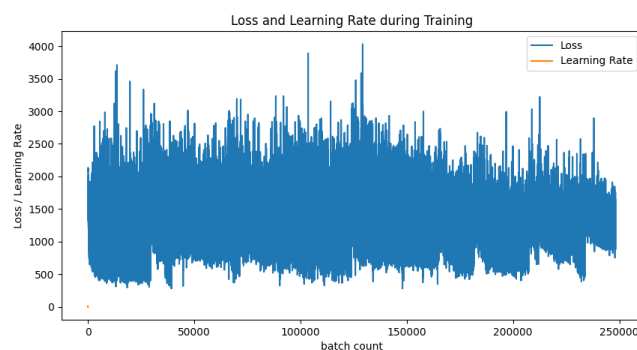


Figure 2. 保持默认设置的 basic result

4.2. 实验分析

第一个消融实验是讨论 batch size 带来增益的同时，我们是否可以找到最优超参配合大规格的 batch size 以达到最优结果，而非局限在局部最优。第二个实验主要是探讨如何改进目前 gpu 低利用率的现状。

4.2.1. 最合适的 batch_size

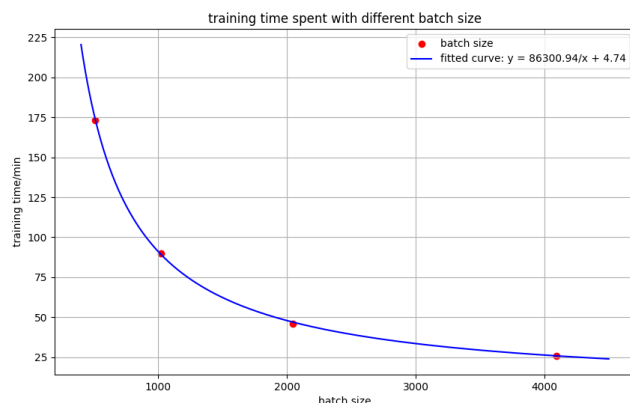
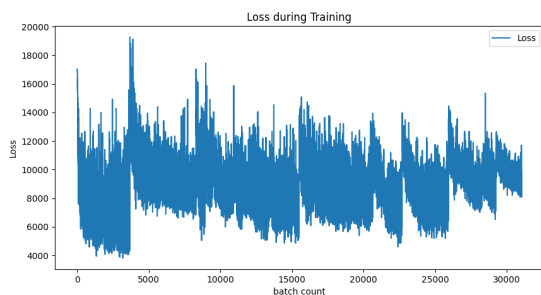
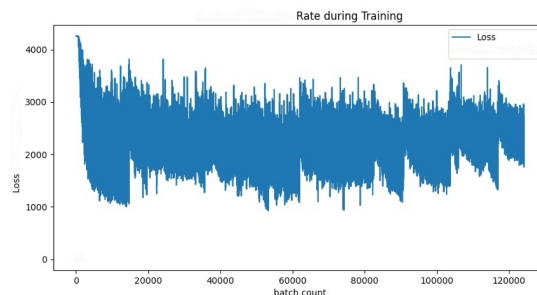


Figure 3. Example figure obtained from PGFPlots

大的 batch size 每次更新用的是更多的样本所以通常会让梯度估计更准确。另外，如 Figure3所示，随着 batch_size 的增大，训练时间明显降低，所以我们希望找到一个充分大的 batch_size 不仅使得训练时间大幅度降低，并且让梯度估计更准确。



(a) 线性增大学习率.



(b) scheduler 用余弦退火

Figure 4. 探究增大 batch size 之后该如何调整超参

然而，如 Figure5所示，只翻倍 batch_size，不改变其他超参的话会梯度爆炸，训练集上的 loss 会在出现剧烈震荡之后指数级增大。

gpu 上更适宜的 adam，最后能够达到 10% 的利用率，还是很低，但是有了很大的改进，之前基本就是 <%1。

5. 附录

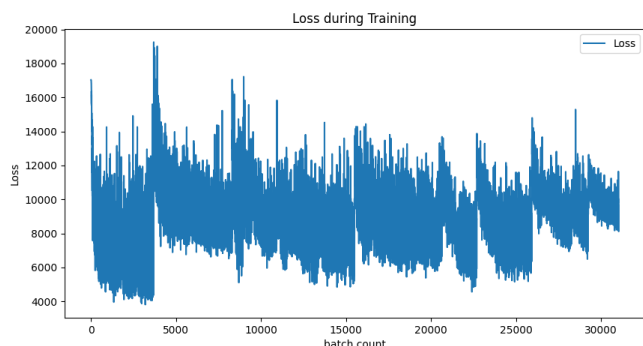


Figure 5. 仅仅提高 batch size 导致的梯度爆炸

首先想到的解决方案是：线性改变初始学习率，但是如 Figure4a(a) 所示，这种方法并不好，于是考虑添加余弦退火，如 Figure4b(b) 所示，这种方法效果很好，可以看到在 batch_size=1024 时，添加了余弦退火的 loss 曲线更加平滑。

4.2.2. gpu 利用率很低

代码特性的问题，发现 gpu 的利用率很低，在 v100 上跑 basic experiment(batch=512,lr=0.005) 时，gpu 的利用率如 Figure6所示，所以训练花费了非常多没必要花费的时间，

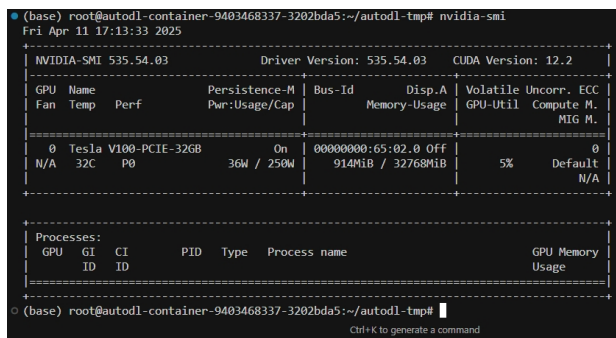


Figure 6. v100 32g 上训练的 gpu 利用率

于是我开始思考怎么能降下去这个训练的时间，其中拉高 batch size，也就是上面的第一个消融实验是第一个尝试的方法，第二个方法就是尽量让矩阵稀疏，把 cudnn 加速打开，sgd 换成

```
Word: 北京
('北京', 1.0)
('中广网', 0.8190181902341008)
('科技', 0.8143659905798679)
('新华网', 0.7759158411514838)
('消息', 0.7732266485457883)
('中新社', 0.7730800374839546)
('张华杰', 0.7728624666098491)
('齐彬', 0.7680992585820394)
('图月', 0.7644391437626)
('周婷玉', 0.7643890601726534)
('星期日', 0.7552879934135533)
('隆重开幕', 0.7536105740209768)
('冯悦', 0.7525122160353124)
('出品商', 0.7503130650979588)
('孙宇挺', 0.7500474026432575)
('李亚杰', 0.7499358132730551)
('中国国际贸易中心', 0.7479358076322907)
('十二月', 0.7474764026414936)
('宴会厅', 0.7467721692560486)
```

Figure 7. ”北京“的上下文

```
Word: 杜兰特
('杜兰特', 0.9999999999999999)
('加身', 0.9623816639542856)
('混血', 0.961314302093499)
('布洛克', 0.9609077344963064)
('科林', 0.9601677136497067)
('最矮', 0.9597962608526671)
('哈尔', 0.9596121851618056)
('叱咤', 0.958970097881563)
('才会赢', 0.9584967068719837)
('哈罗德', 0.9580362641865104)
('达科', 0.9578777651572472)
('高盟杰', 0.9576383071225824)
('皮尔斯', 0.9575541394701027)
('梅尔', 0.9573721128286733)
('荒野', 0.9570977076795724)
('奎尔', 0.9570763437132992)
('哈金斯', 0.9570701447311536)
('好手', 0.9570351138110267)
('引以为傲', 0.9569280744009105)
('霹雳', 0.9567710347965485)
```

Figure 8. ”杜兰特“的上下文